Generics

Bib: Tutoriais Java:

https://docs.oracle.com/javase/tutorial/java/generics/index.html

https://docs.oracle.com/javase/tutorial/extra/generics/index.html

Programação III (PG3)

João Pedro Patriarca, (joao.patriarca@isel.pt),

Gabinete F.0.23 do edifício F, LEETC, ADEETC

ISEL

Resumo

- Introdução
- Classes/interfaces genéricas
- Métodos genéricos
- Estabelecimento de delimitadores nos parâmetros tipo
- Restrições nos genéricos

Resumo

- Introdução
- Classes/interfaces genéricas
- Métodos genéricos
- Estabelecimento de delimitadores nos parâmetros tipo
- Restrições nos genéricos

Introdução

- *Generics* foi introduzido a partir da versão JavaSE 5
- Permite a definição de classes, interfaces ou métodos genéricos com *parâmetros tipo*, ou seja, tipos que não estão especificados
  - Ex: `public class ArrayList<E> { ... }`
  - O *parâmetro tipo* é especificado, posteriormente, na utilização do tipo genérico
  - Ex: `ArrayList<Integer> list;`
- Permite que a mesma classe, interface ou método genérico possa ser usado com diferentes tipos
  - O tipo genérico não é expandido para os tipos concretos que o usa; nem ao nível do código fonte, nem ao nível do código binário; gera um único *.class* depois de compilado
- Antes da introdução de genéricos era usado o tipo `Object`
- Benefícios:
  - *Strong type check*: verificação em tempo de compilação na utilização adequada do tipo de dados
  - Eliminação do uso extensivo de *casts* em métodos que retornem um *parâmetro tipo*
  - Facilita a escrita de algoritmos genéricos que atuam sobre diferentes tipos de dados

Parâmetros tipo

- Por convenção, a declaração de um *parâmetro tipo* é feita com uma única letra maiúscula
  - **T** para *Type*; **E** para *Element*; **N** para *number*; **K** para *key*; **V** para *Value*
  - Se a classe, interface ou método genérico usar diferentes *parâmetros tipo*, declaram-se várias letras separadas por ‘,’: **<K, V>**
- Um *parâmetro tipo* pode ser concretizado com qualquer classe ou interface
- A classe ou interface usada para especificar um *parâmetro tipo* denomina-se por *argumento tipo*
- Não é permitido usar tipos primitivos para definir *argumentos tipo* – tem de se usar *wrappers* dos tipos primitivos

Classes *wrapper*

- Para cada tipo primitivo existe uma classe *wrapper* correspondente

Tipo primitivo	Classe *wrapper*	Métodos relevantes das classe *wrapper*

`boolean`	`Boolean`	Tipo wrapper -> tipo primitivo
		`primitive_type <primitive_type>Value();`
		String -> tipo primitivo
		`static primitive_type parse<wrapper_type>(String);`
		Tipo primitivo -> tipo wrapper
		`static wrapper_type valueOf(primitive_type);`

`byte`	`Byte`	
`short`	`Short`	
`int`	`Integer`	
`long`	`Long`	
`float`	`Float`	
`double`	`Double`	
`char`	`Character`	

- Conversão automática de tipo primitivo para a classe *wrapper*: *box*
- Conversão automática de classe *wrapper* para tipo primitivo: *unbox*

Resumo

- Introdução
- Classes/interfaces genéricas
- Métodos genéricos
- Estabelecimento de delimitadores nos parâmetros tipo
- Restrições nos genéricos

Classes e interfaces genéricas versus `Object`

- Uma classe ou interface genérica é um *tipo genérico* parametrizado por outros tipos classe/interface

```
// Tipo genérico mas não parametrizado
// Usa Object para conseguir a generalização
public class BoxObject {
   private Object t;
   public BoxObject() {}
   public BoxObject(Object t)
   { this.t = t; }
   public Object get() { return t; }
   public void set(Object t) { this.t = t; }
}
```

```
// Tipo genérico parametrizado
// Na utilização, é especificado o tipo concreto
// que serve como argumento tipo
public class BoxGeneric<T> {
   private T t;
   public BoxGeneric() {}
   public BoxGeneric(T t)
   { this.t = t; }
   public T get() { return t; }
   public void set(T t) { this.t = t; }
}
```

Se a classe envolver múltiplos parâmetros tipo, declara-se class Name<T1, ..., Tn> { ... }

Utilização de uma classe genérica versus `Object`

```
public class BoxDemo {
    public static void main(String[] args) {
        BoxObject b1 = new BoxObject();
        BoxGeneric<String> b2 = new BoxGeneric<>();

        b1.set("123");
        String s1 = (String)b1.get();
        b2.set("123");
        String s2 = b2.get();
        b1.set(123);
        //b2.set(123);
        String s3 = (String)b1.get();
    }
}
```

A partir da versão7, suporta a omissão do *argumento tipo* se o compilador conseguir inferir qual é o tipo. Antes da versão 7 tinha que se escrever `new BoxGeneric<String>();`

Necessário *cast*

Sem necessidade de *cast*

Não produz erro de compilação: o inteiro 123 é *autoboxed* para Integer que é Object

Erro de compilação: o inteiro 123 não é convertível em *string*

Produz a exceção `ClassCastException`, apenas durante a execução

Exemplo com múltiplos parâmetros tipo

```
public interface Pair <K, V> {
    public K getKey();
    public V getValue();
}
```

```
public class OrderedPair <K, V>
implements Pair <K, V> {
    private K key;
    private V value;
    public OrderedPair(K key, V value) {
        this.key = key;    this.value = value;
    }
    public K getKey()   { return key; }
    public V getValue() { return value; }
}
```

```
public class OrderedPairDemo {
    public static void main(String[] args) {
        Pair<String, Integer> p1 =
                new OrderedPair<>("Even", 8);
        Pair<String, String>  p2 =
                new OrderedPair<>("hello", "world");
        Pair<Integer, List<String>> p3 =

                new OrderedPair<>(
                    1,
                    Arrays.asList("abc", "def")
                );
} }
```

Um *argumento tipo* pode ser outro tipo genérico parametrizado

Mais sobre genéricos

- Uma classe genérica é sempre a mesma classe, independentemente dos tipos que se usem quando se instancia objetos da classe

```
ArrayList<Integer> iList = new ArrayList<>();
ArrayList<String> sList = new ArrayList<>();
boolean sameClasse = iList.getClass() == sList.getClass();
// produz true
```

- A definição da classe `ArrayList<E>`, quando compilada com o parâmetro tipo, dá origem a um único *.class*
- Portanto, `ArrayList<Integer>` ou `ArrayList<String>` são a mesma classe

Uso de tipos genéricos não parametrizados (*raw types*)

- A utilização de tipos genéricos não parametrizados deve evitar-se!

```
static void rawParametrizedType() {
    BoxGeneric<String> b1 = new BoxGeneric<>();
    BoxGeneric b2 = b1;

    b2.set(123);

    BoxGeneric b3 = new BoxGeneric();
    BoxGeneric<String> b4 = b3;

    // BoxGeneric<Object> b5 = b1;
}
```

O compilador não produz qualquer erro ou aviso, para efeitos de compatibilidade com versões do Java anteriores aos genéricos. A variável `b2` passa a ser usada como se fosse um `BoxObject` (portanto, perdeu-se todos os benefícios dos genéricos)

Nem gera qualquer erro de compilação ao usar-se um inteiro em vez de uma *string*

O inverso já não se verifica. O compilador produz o aviso *unchecked convertion*

O compilador gera erro de compilação, mesmo `String` derivando de `Object`. De notar que `BoxGeneric<String>` não deriva de `BoxGeneric<Object>`

Resumo

- Introdução
- Classes/interfaces genéricas
- Métodos genéricos
- Estabelecimento de delimitadores nos parâmetros tipo
- Restrições nos genéricos

Métodos genéricos

- Uma classe genérica ou parametrizada, por norma, contém campos que são direta ou indiretamente relacionados com os parâmetros tipo
- Um método genérico introduz os seus próprios parâmetros tipo
- Os parâmetros tipo de um método genérico pertencem apenas ao método
- É permitido usar parâmetros tipo em métodos de instância, estáticos ou construtores

```
public class GenericMethod {
   public static <K, V> boolean compare( Pair<K, V> p1, Pair<K, V> p2 ) {
      return p1.getKey().equals(p2.getKey()) && p1.getValue().equals(p2.getValue());
} }
```

```
Pair<Integer, String> p1 = new OrderedPair<>(1, "apple");
Pair<Integer, String> p2 = new OrderedPair<>(2, "pear");
boolean same1 = GenericMethod.<Integer, String>compare(p1, p2);
boolean same2 = GenericMethod.compare(p1, p2);
```

Por inferência de tipos, não é preciso especificar os argumentos tipo, tal como na definição de same1

Inferência de tipos (1 de 2)

- O compilador infere o tipo de argumento mais específico que serve a chamada sem erros, em termos de tipos

```
public static <T> void fromArrayToCollection(T[] a, Collection<T> c) {
   for(T o: a)
      c.add(o);
}
public static void fromArrayToCollectionDemo() {
   Object[] oa = new Object[100];
   Collection<Object> co = new ArrayList<Object>();
   fromArrayToCollection(oa, co); // T inferred to be Object
   String[] sa = new String[100];
   Collection<String> cs = new ArrayList<String>();
   fromArrayToCollection(sa, cs); // T inferred to be String
   fromArrayToCollection(sa, co); // T inferred to be Object
   // continua
```

Inferência de tipos (2 de 2)

- O compilador infere o tipo de argumento mais específico que serve a chamada sem erros, em termos de tipos

```
public static void fromArrayToCollectionDemo() {
   Collection<Object> co = new ArrayList<Object>();
   Collection<String> cs = new ArrayList<String>();
   // continuação
   Integer[] ia = new Integer[100];
   Float[] fa = new Float[100];
   Number[] na = new Number[100];
   Collection<Number> cn = new ArrayList<Number>();
   fromArrayToCollection(ia, cn);    // T inferred to be Number
   fromArrayToCollection(fa, cn);    // T inferred to be Number
   fromArrayToCollection(na, cn);    // T inferred to be Number
   fromArrayToCollection(na, co);    // T inferred to be Object
   // fromArrayToCollection(na, cs); // compile error: Number and String are not related
}
```

Resumo

- Introdução
- Classes/interfaces genéricas
- Métodos genéricos
- Estabelecimento de delimitadores nos parâmetros tipo
- Restrições nos genéricos

Relação de herança versus genéricos

- Visto anteriormente:

```
static void inheritanceVersusGenerics() {
    BoxGeneric<Integer> b1 = new BoxGeneric<>(123);
    // BoxGeneric<Number> b2 = b1; // Erro de compilação
}
```

O compilador gera erro de compilação, mesmo `Integer` derivando de `Number`. De notar que, embora `Integer` derive de `Number`, `BoxGeneric<Integer>` não deriva de `BoxGeneric<Number>`

Relação de herança versus genéricos

- Visto anteriormente:

```
static void inheritanceVersusGenerics() {
   BoxGeneric<Integer> b1 = new BoxGeneric<>(123);
   // BoxGeneric<Number> b2 = b1; // Erro de compilação
}
```

O compilador gera erro de compilação, mesmo `Integer` derivando de `Number`. De notar que, embora `Integer` derive de `Number`, `BoxGeneric<Integer>` não deriva de `BoxGeneric<Number>`

- Não será um pouco limitativo?!? O tipo `Integer` estende `Number`
- Por outro lado, pode ser igualmente conveniente limitar o domínio de tipos que se podem usar como parâmetro tipo

Especificação de delimitadores no parâmetro tipo

- Por vezes é conveniente limitar o domínio dos parâmetros tipo que se podem usar num tipo parametrizado

```
public class BoundedParameterType<T extends Number> {
   private T e1, e2;
   public BoundedParameterType(T e1, T e2) {  this.e1 = e1; this.e2 = e2;  }
   public double sum() { return e1.doubleValue() + e2.doubleValue(); }   }
```

A limitação do parâmetro tipo T a `Number` e subclasses, permite chamar métodos definidos na classe `Number`

Sem delimitadores não seria possível usar um método de `Number`

- *<T extends Number>* significa que apenas se pode parametrizar com o tipo `Number` ou classes descendentes (entenda-se *extends* como *estenda* ou *implemente*)

```
public static void main(String[] args) {
   BoundedParameterType<Integer> bv = new BoundedParameterType<>(10, 26);
   double sum = bv.sum();
}
```

O compilador gera erro se usar-se outro tipo que não `Number` ou tipos descendentes

Múltiplos delimitadores nos parâmetros tipo

- É possível declarar múltiplo limites no parâmetro tipo

  `<T extends TypeA & TypeB & TypeC>`

- O argumento tipo terá de ser um subtipo de todos os tipos enumerados depois da cláusula *extends*
- Os tipos enumerados podem ser classes ou interfaces
  - Na presença de classes e interfaces em simultâneo, a classe têm de ser declarada em primeiro

```
class A { ... }
interface B { ... }
interface C { ... }
class D <T extends A & B & C> { ... }
```

Delimitadores nos parâmetros tipo dos métodos genéricos

- Tal como num tipo parametrizado, também permite limitar os parâmetros tipo em métodos genéricos

```
public class BoundedParameterTypeInGenericMethod {
   public static <T extends Comparable<T>> int countGreaterThan(T elem, T ... sequence) {
      int count = 0;
      for (T e : sequence)
         if (e.compareTo(elem) > 0)  ++count;
      return count;
   }
   public static void main(String[] args) {
      int cnt = countGreaterThan(10.2, 1.0, 2.0, 3.0, 4.5, 10.2, 10.5, 10.0, 20.0);
      System.out.println(cnt);
} }
```

```
2
```

Delimitadores superiores no parâmetro tipo Problema

- De notar que:

```
class A <T extends Number> { ... }
```

- Permite

`new A<Number>()` ou **`new A<Integer>()`** ou **`new A<Double>()`**

- Porque os tipos `Integer` e `Double` são `Number`
- Já se for:

```
class B <T extends List<Number>> { ... }
```

- Escrever

```
new B<List<Integer>>();
```

- Produz erro de compilação porque `List<Integer>` não é um subtipo de `List<Number>`

Delimitadores superiores no parâmetro tipo

Solução – utilização de *wildcard* (*qualquer tipo* ou *tipo desconhecido*)

- Usa-se *wildcard* com limite superior
  - Em código genérico o *wildcard* é representado pelo caractere '?'
  - *wildcard* representa *qualquer tipo* ou *tipo desconhecido*

```
public class UpperBoundedWildcardDemo {
   public static double sumOfList(List<? extends Number> list) {
      double s = 0.0;
      for (Number n : list) s += n.doubleValue();
      return s;
   }
   public static void main(String[] args) {
      List<Integer> li = Arrays.asList(1, 2, 3);  System.out.println("sum = " + sumOfList(li));
      List<Double> ld = Arrays.asList(1.6, 2.3, 3.5);  System.out.println("sum = " + sumOfList(ld));
} }
```

Permite usar como argumento tipo `List` de `Number` ou `List` de descendentes de `Number`

```
sum = 6.0
sum = 7.4
```

Wildcard ilimitado

- Considerando o exemplo `List<?>`
  - Denominado por *lista de elementos de qualquer tipo*
- Existem dois cenários onde a utilização de *wildcard* ilimitado é útil
  - Quando a implementação do método usa apenas funcionalidades da classe `Object`
  - Quando se usam métodos do tipo genérico sem precisar de conhecer o parâmetro tipo. Ex: `List.size()` ou `List.clear()`

```
public class UnboundedWildcardDemo {
   public static void printList(List<Object> list) {
      for (Object elem: list)  System.out.print(elem + " ");
      System.out.println();
   }
   public static void main(String ... args) {
      List<Integer> li = Arrays.asList(1, 2, 3);    List<String>  ls = Arrays.asList("one", "two", "three");
      //printList(li);   printList(ls);
} }
```

Erro de compilação, novamente, porque `List<Integer>` e `List<String>` não são `List<Object>`

Wildcard ilimitado

- Considerando o exemplo `List<?>`
  - Denominado por *lista de elementos de qualquer tipo*
- Existem dois cenários onde a utilização de *wildcard* ilimitado é útil
  - Quando a implementação do método usa apenas funcionalidades da classe `Object`
  - Quando se usam métodos do tipo genérico sem precisar de conhecer o parâmetro tipo. Ex: `List.size()` ou `List.clear()`

```
public class UnboundedWildcardDemo {
   public static void printList(List<?> list) {
      for (Object elem: list)  System.out.print(elem + " ");
      System.out.println();
   }
   public static void main(String ... args) {
      List<Integer> li = Arrays.asList(1, 2, 3);    List<String>  ls = Arrays.asList("one", "two", "three");
      printList(li);   printList(ls);
} }
```

Definindo o parâmetro como uma lista de elementos de *qualquer tipo*, permite usar `List<Integer>` ou `List<String>`

```
1 2 3
one two three
```

Regra para decidir entre ou método genérico ou *wildcard*

- Cada par de métodos é equivalente. Qual assinatura se deve usar?

```
public static double sumOfList1(List<? extends Number> list) { ... }
public static <T1 extends Number> double sumOfList2(List<T1> list) { ...}

public static void printList1(List<?> list) { ... }
public static <T2> void printList2(List<T2> list) { ... }

public static <T3> void copy1(List<T3> dest, List<? extends T3> src) { ... }
public static <T3, S extends T3> void copy2(List<T3> dest, List<S> src) { ... }
```

- Por norma, sempre que exista uma única referência ao tipo genérico na assinatura do método (parâmetros e retorno), deve usar-se *wildcard*
  - Porque é mais conciso, mais claro e mais flexível
  - Porque permite ser usado fora da assinatura do método (i.e. campos da classe) sem ter de declarar o tipo genérico ao nível da classe
  - Estão nestas condições T1, T2 e S

Outro exemplo observando a API da interface `Collection`

// Usando wildcard	// Usando métodos genéricos
interface Collection<E> {	interface Collection<E> {
public boolean containsAll (Collection<?> c);	public <T> boolean containsAll (Collection<T> c);
public boolean addAll (Collection<? extends E> c);	public <T extends E> boolean addAll (Collection<T> c);
}	}

- Não existe dependências entre o tipo de retorno e o parâmetro tipo **T**, nem dependências com o tipo de outros parâmetros (o parâmetro **T** é usado apenas num único sítio)
- O objetivo do parâmetro tipo **T** em vez de **E** é permitir a invocação deste métodos com coleções com elementos de tipos diferentes de **E**

- Conclusão: deve ser usado *wildcard* pois os *wildcards* foram desenhados para suportar subtipificação mais flexível
- Métodos genéricos permitem que parâmetros tipos sejam usados para expressar dependências entre tipos de um ou mais argumentos de um método e/ou tipo de retorno. Não existindo uma dependência deste género, o método genérico não deve ser usado

Wildcard delimitado inferiormente

- O objetivo corresponde a restringir *qualquer tipo* a um tipo específico ou a um supertipo do tipo específico (*<? super SpecificType>*)
  - Inverso ao "qualquer tipo delimitado superiormente": restringe *qualquer tipo* a um tipo específico ou a um subtipo (*<? extends SpecificType>*)

```
public class LowerBoundedWildcardDemo {
   public static void addRange(List<? super Integer> list, int n)
   { for(int i = 1; i <= n; i ++) list.add ( i ); }
   public static void main(String[] args) {
      List<Integer> li = new ArrayList<>(Arrays.asList(1, 3, 4, 10));
      List<Number> ln = new ArrayList<>(Arrays.asList(1.7, 3.5, 4, 10.2));
      addRange(li, 10);   addRange(ln, 5);
} }
```

- Não é possível especificar em simultâneo *qualquer tipo* delimitado inferiormente e superiormente
- Um delimitador inferior apenas compila com *wildcard*

Linhas orientadoras para utilização de *wildcards*

- Quando se deve usar *<? extends Type>* ou quando se deve usar *<? super Type>*?
- Auxilia o raciocínio pensar no objeto como *produtor* ou como *consumidor* de valores:
  - *Produtor*: o objeto fornece/produz os dados que são usados pelo código do método. Por exemplo, para uma lista, usam-se operações de *get* (consulta)
  - *Consumidor*: o objeto guarda os dados produzidos pelo código do método. Por exemplo, para uma lista, usam-se operações de *set* (alteração do estado)
  - *Produtor*/*Consumidor*: atenção porque pode fornecer e guardar dados, em simultâneo

  Ex: `void copy(List<? super T> dst, List<? extends T> src)`

  `src`: *produtor* (fornece/produz os dados)

  `dst`: *consumidor* (guarda/consome os dados)
- Linhas orientadoras:
  - Usar um *wildcard* delimitado superiormente (*<? extends Type>*) no caso de um *produtor*
  - Usar um *wildcard* delimitado inferiormente (*<? super Type>*) no caso de um *consumidor*
  - Usar um *wildcard* ilimitado (*<?>*) no caso do objeto ser acedido apenas com métodos da classe `Object`
  - Não usar *wildcard* no caso do objeto ser *produtor* e *consumidor*, em simultâneo

Linhas orientadoras para utilização de *wildcards*
Exemplo

```
public class PecsDemo { // Producer Extends, Consumer Super
   public static void pe(List<? extends Number> list) {
      //list.set(0, 10.0); // Erro: os elementos podem não ser do tipo Double (Integer, por exemplo)
      Number v = list.getFirst();
      System.out.println(v);
   }
   public static void cs(List<? super Double> list) {
      list.set(0, 10.0); // Suporta Double ou qualquer supertipo
      //Double d = list.getFirst(); // Erro: os elementos podem ser de um supertipo de Double
      Object d = list.getFirst(); // O único supertipo compatível é Object (que não produz erro)
      System.out.println(d);
   }
   public static void copy(List<? super Number> dst, List<? extends Number> src) {
      for (Number n: src)
         dst.add(n);
} }
```

Resumo

- Introdução
- Classes/interfaces genéricas
- Métodos genéricos
- Estabelecimento de delimitadores nos parâmetros tipo
- Restrições nos genéricos

Restrições na utilização de genéricos – não se pode ...

- instanciar tipos genéricos com tipos primitivos: new ArrayList<int>();
- criar instâncias de parâmetros tipo: new T(); ou new T[10];
- declarar campos estáticos cujos tipos são parâmetros tipo
- usar *cast* ou *instanceof* com tipos parametrizados
- criar *arrays* de tipos parametrizados
- criar, capturar (*catch*) ou lançar (*throws*) objetos do tipo tipos parametrizados
- sobrecarregar (*overload*) um método cuja diferença se limite ao parâmetro tipo